COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 27, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 3.1 Linked Lists

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include "simply_LL.h"

int main (){

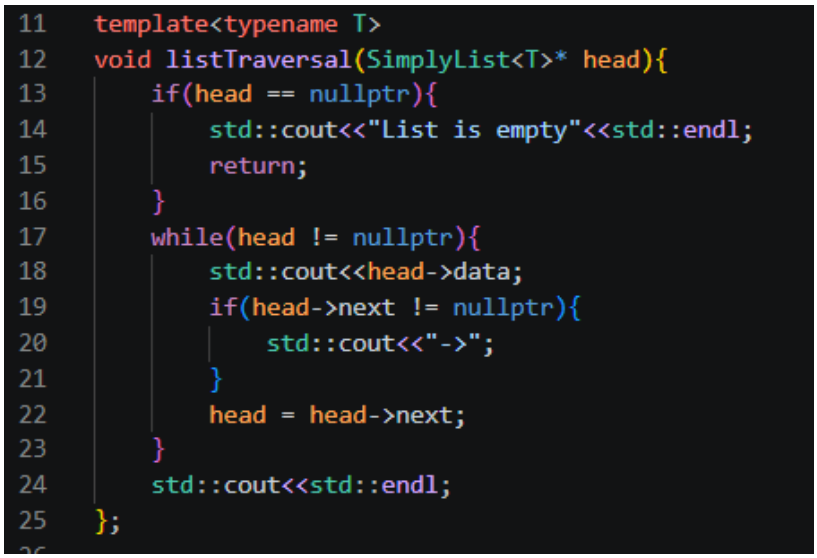
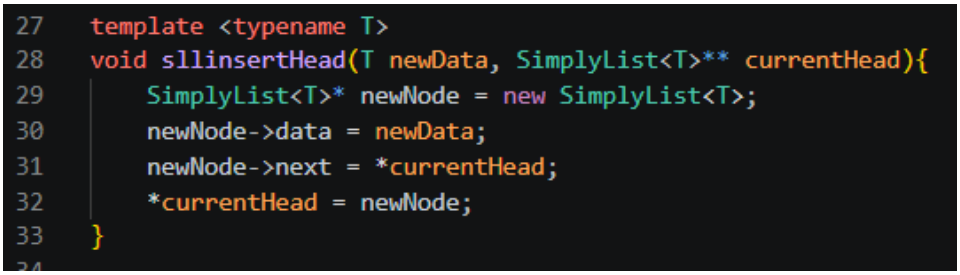
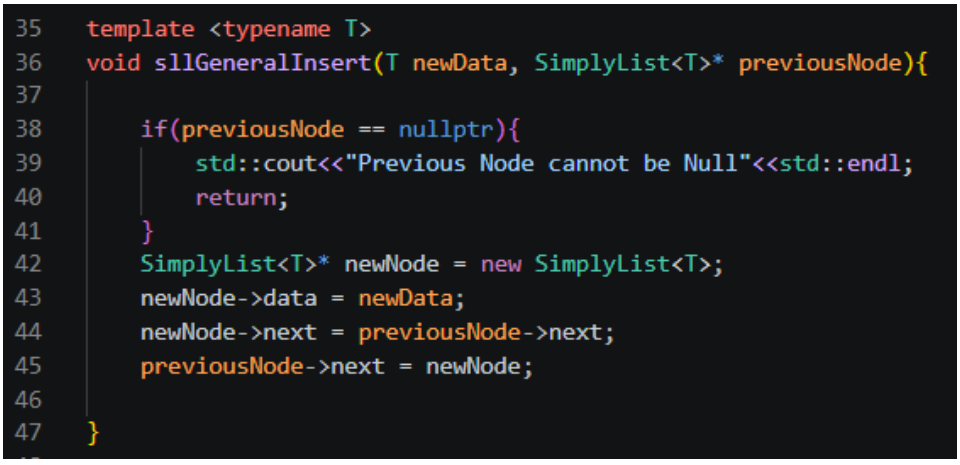
    //creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second =  new SimplyList<char>;
    SimplyList<char>* third =  new SimplyList<char>;
    SimplyList<char>* fourth =  new SimplyList<char>;

    //assign the data:
    head-> data = 'S';
    head->next = second;
    second->data = 'A';
    second->next = third;
    third->data = 'Y';
    third->next = fourth;
    fourth->data = 'A';
    fourth->next = nullptr;
```

Discussion

The initial implementation contains 4 nodes with each one containing data and points to the next node. The linked list has a head and the last node points to a nullptr. This is done by first creating a new object for each node and setting their data and pointing them to next using the arrow (→).

Table 3-1. Output of Initial/Simple Implementation

Operations	Screenshot
Traversal	
Insertion at head	
Insertion at any part of the list	

```
11  template<typename T>
12  void listTraversal(SimplyList<T>* head){
13      if(head == nullptr){
14          std::cout<<"List is empty"<<std::endl;
15          return;
16      }
17      while(head != nullptr){
18          std::cout<<head->data;
19          if(head->next != nullptr){
20              std::cout<<"->";
21          }
22          head = head->next;
23      }
24      std::cout<<std::endl;
25  };
```

```
27  template <typename T>
28  void sllinsertHead(T newData, SimplyList<T>** currentHead){
29      SimplyList<T>* newNode = new SimplyList<T>;
30      newNode->data = newData;
31      newNode->next = *currentHead;
32      *currentHead = newNode;
33  }
```

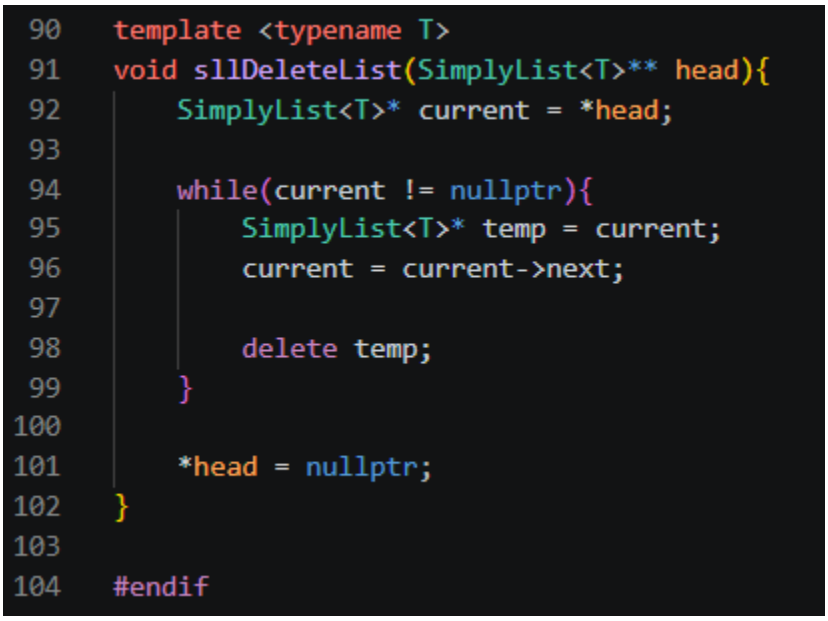
```
35  template <typename T>
36  void sllGeneralInsert(T newData, SimplyList<T>* previousNode){
37
38      if(previousNode == nullptr){
39          std::cout<<"Previous Node cannot be Null"<<std::endl;
40          return;
41      }
42      SimplyList<T>* newNode = new SimplyList<T>;
43      newNode->data = newData;
44      newNode->next = previousNode->next;
45      previousNode->next = newNode;
46
47  }
```

Insertion at the end

```
49  template <typename T>
50  void sllInsertEnd(T newData, SimplyList<T>** head){
51      SimplyList<T>* newNode = new SimplyList<T>;
52      SimplyList<T>* currentNode = *head;
53      newNode->data = newData;
54      newNode->next = nullptr;
55      while(currentNode->next != nullptr){
56          currentNode = currentNode->next;
57      };
58
59      currentNode->next = newNode;
60  }
```

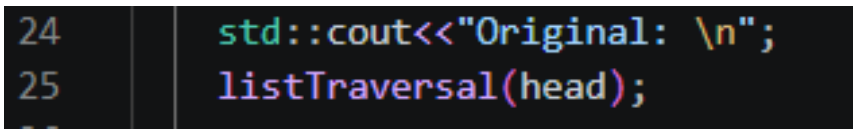
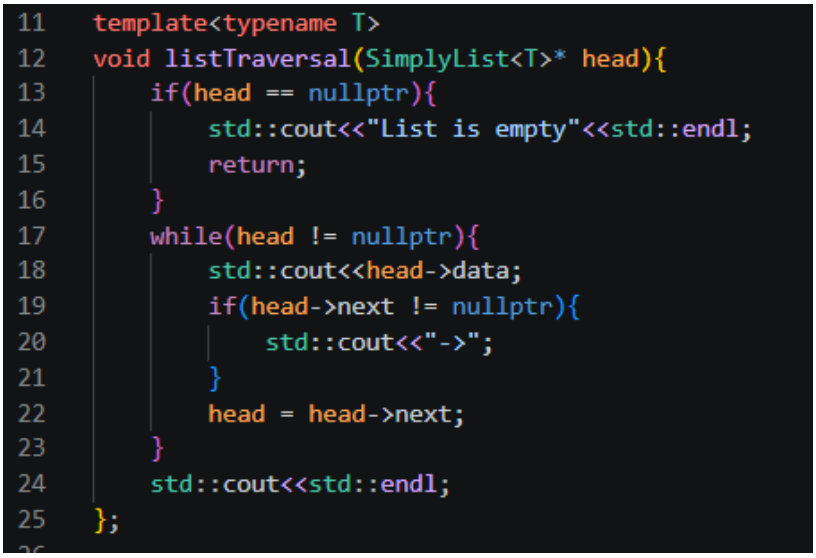
Deletion of a node

```
62  template <typename T>
63  void sllDeleteNode(T findData, SimplyList<T>** head){
64      if(*head == nullptr) return;
65
66      SimplyList<T>* currNode = *head;
67      SimplyList<T>* prevNode = nullptr;
68
69      //searching loop
70      while(currNode != nullptr && currNode->data != findData){
71          prevNode = currNode;
72          currNode = currNode->next;
73      }
74
75      //data was found
76      if (currNode == nullptr) return;
77
78      //what if the node to be deleted is the head node
79      if(prevNode == nullptr){
80          *head = currNode->next;
81      }
82      else{
83          prevNode->next = currNode->next;
84      };
85      //we have to delete the node?
86
87      delete currNode;
88  }
```

Delete list	

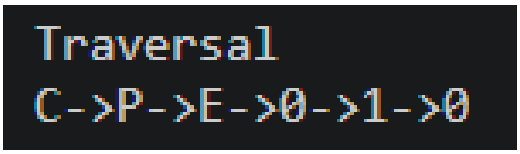
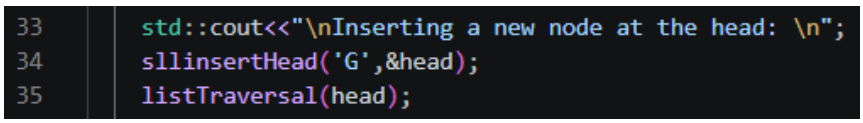
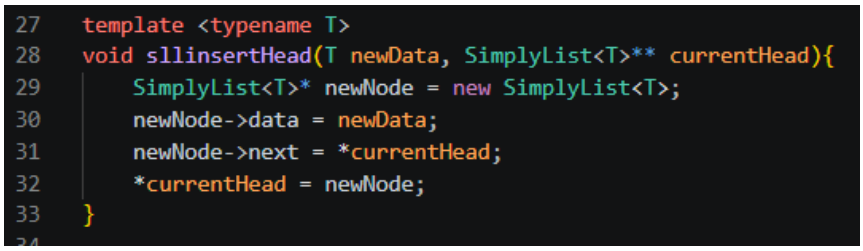
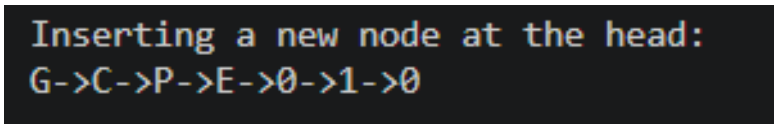
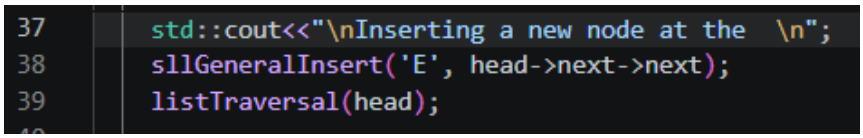
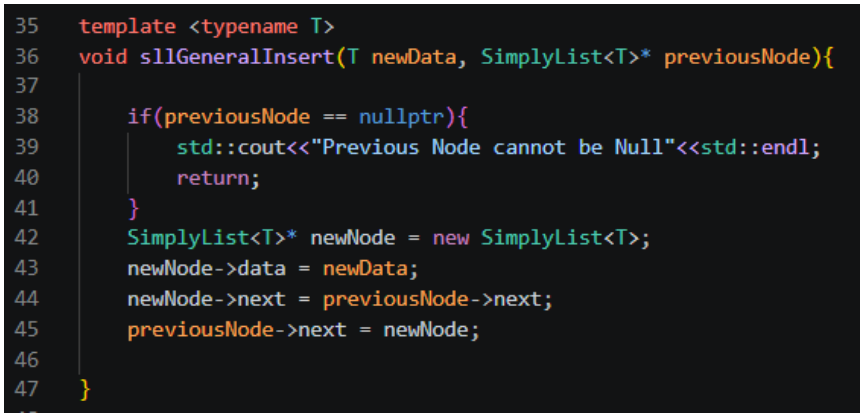
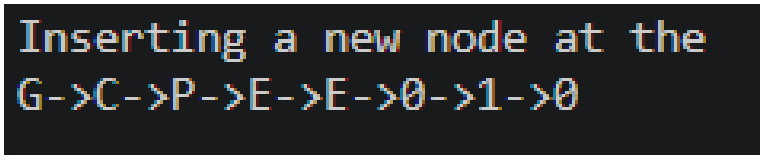
```
90  template <typename T>
91  void sllDeleteList(SimplyList<T>** head){
92      SimplyList<T>* current = *head;
93
94      while(current != nullptr){
95          SimplyList<T>* temp = current;
96          current = current->next;
97
98          delete temp;
99      }
100
101     *head = nullptr;
102 }
103
104 #endif
```

Table 3-2. Code for the List Operations

a.	**Source Code**	
	Console	

```
24      std::cout<<"Original: \n";
25      listTraversal(head);
```

```
11  template<typename T>
12  void listTraversal(SimplyList<T>* head){
13      if(head == nullptr){
14          std::cout<<"List is empty"<<std::endl;
15          return;
16      }
17      while(head != nullptr){
18          std::cout<<head->data;
19          if(head->next != nullptr){
20              std::cout<<"->";
21          }
22          head = head->next;
23      }
24      std::cout<<std::endl;
25  };
```

	Output	
b.	**Source Code**	
	Console	
	Output	
c.	**Source Code**	
	Console	
	Output	

d.

Source Code

```
47      std::cout<<"\nDeleting a node \n";
48      sllDeleteNode('C',&head);
49      listTraversal(head);
```

Console

```
62  template <typename T>
63  void sllDeleteNode(T findData, SimplyList<T>** head){
64      if(*head == nullptr) return;
65
66      SimplyList<T>* currNode = *head;
67      SimplyList<T>* prevNode = nullptr;
68
69      //searching loop
70      while(currNode != nullptr && currNode->data != findData){
71          prevNode = currNode;
72          currNode = currNode->next;
73      }
74
75      //data was found
76      if (currNode == nullptr) return;
77
78      //what if the node to be deleted is the head node
79      if(prevNode == nullptr){
80          *head = currNode->next;
81      }
82      else{
83          prevNode->next = currNode->next;
84      };
85      //we have to delete the node?
86
87      delete currNode;
88  }
```

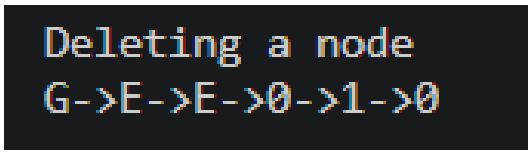
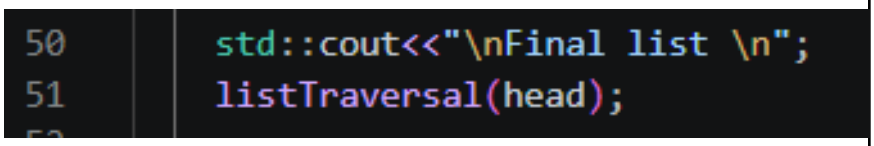
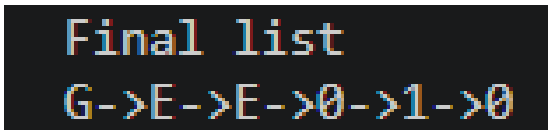
Output

```
Deleting a node
G->P->E->E->0->1->0
```

e.

Source Code

```
45      std::cout<<"\nDeleting a node \n";
46      sllDeleteNode('P',&head);
47      listTraversal(head);
```

	Console	
	Output	
f.	**Source Code**	
	Console	
	Output	

```template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){
    if(*head == nullptr) return;

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    //searching loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //data was found
    if (currNode == nullptr) return;

    //what if the node to be deleted is the head node
    if(prevNode == nullptr){
        *head = currNode->next;
    }
    else{
        prevNode->next = currNode->next;
    };
    //we have to delete the node?

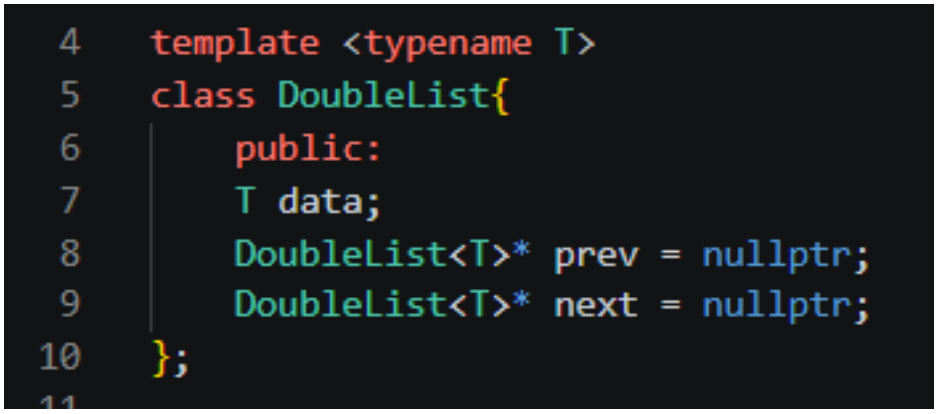
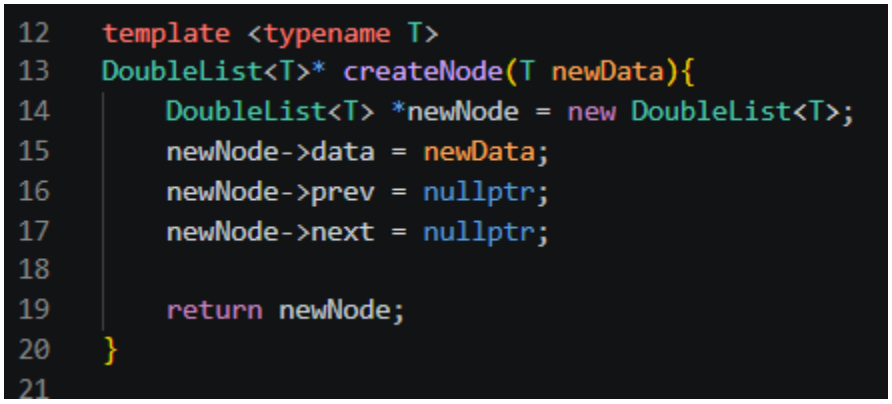
    delete currNode;
}
```

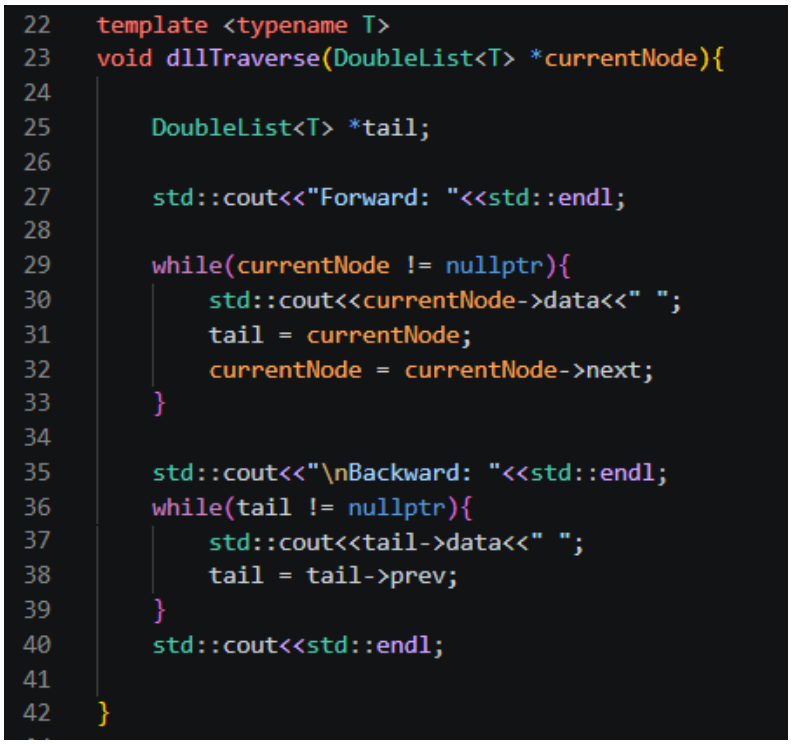
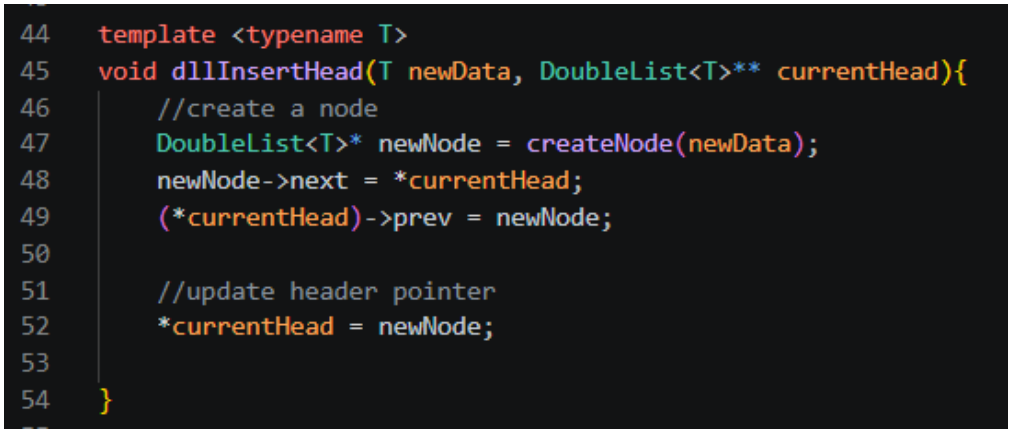
```
Deleting a node
G->E->E->0->1->0
```

```
    std::cout<<"\nFinal list \n";
    listTraversal(head);
```

```
Final list
G->E->E->0->1->0
```

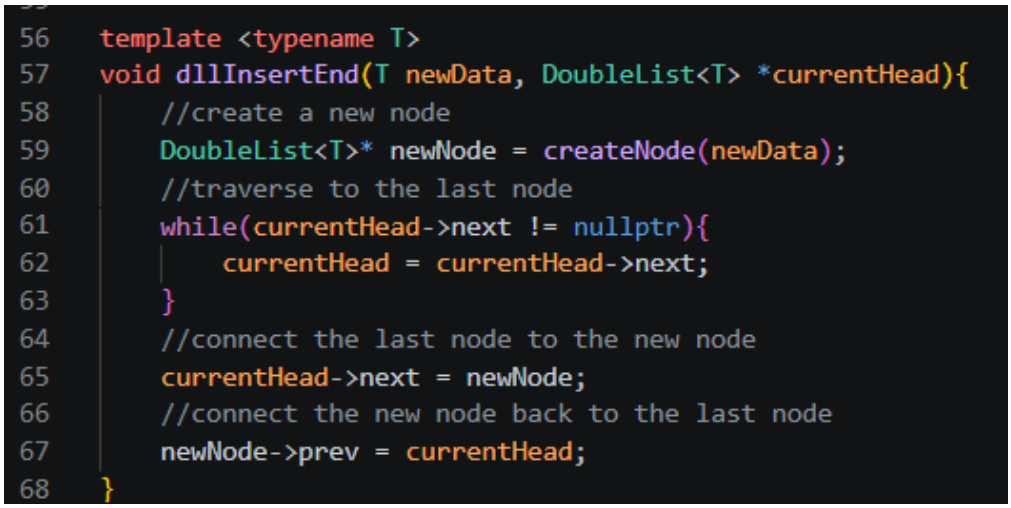
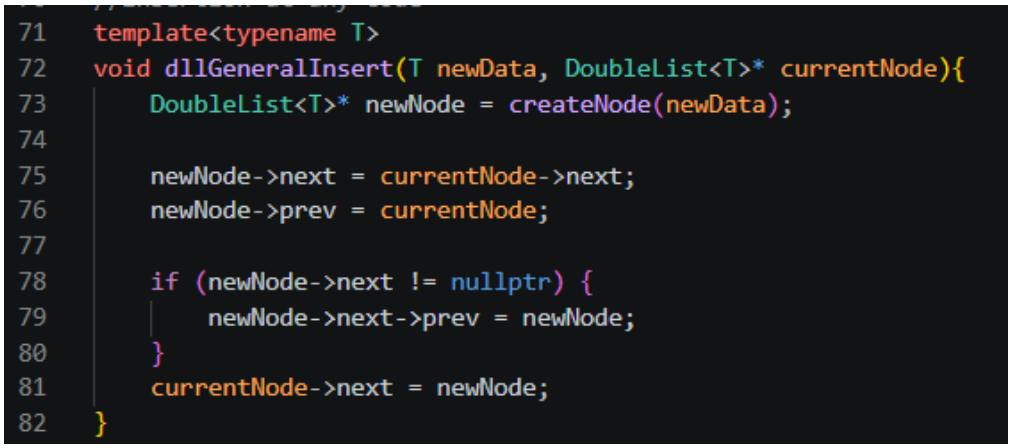
Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot	
Discussion	The public class of the DoubleList includes a new variable called prev along with the data and next variables. Both of the prev and next pointer point to null.
Screenshot	
Discussion	This is a new operation for creating a node in a doubly linked list. SInce each nodes points to the next node and to the previous, we have to make it so that the node points to next and to the prev. Then, we can assign which node is their next and previous while also assigning their data.

Screenshot	
Discussion	For the traverse operation, instead of going to each node in one direction, the program also needs to display the other way. We do this by setting the tail to the current node that is being displayed, then we display that in another while loop.
Screenshot	
Discussion	For the inserting in the head operation, we simply create a new node. Then, we set the next node of that new node to the address of the current head. Then, the previous node of the current head to the new node. Then, we set the current head to the new node, making it the head.

```
22  template <typename T>
23  void dllTraverse(DoubleList<T> *currentNode){
24
25      DoubleList<T> *tail;
26
27      std::cout<<"Forward: "<<std::endl;
28
29      while(currentNode != nullptr){
30          std::cout<<currentNode->data<<" ";
31          tail = currentNode;
32          currentNode = currentNode->next;
33      }
34
35      std::cout<<"\nBackward: "<<std::endl;
36      while(tail != nullptr){
37          std::cout<<tail->data<<" ";
38          tail = tail->prev;
39      }
40      std::cout<<std::endl;
41
42  }
```

```
44  template <typename T>
45  void dllInsertHead(T newData, DoubleList<T>** currentHead){
46      //create a node
47      DoubleList<T>* newNode = createNode(newData);
48      newNode->next = *currentHead;
49      (*currentHead)->prev = newNode;
50
51      //update header pointer
52      *currentHead = newNode;
53
54  }
```

Screenshot	
Discussion	The inserting in the end operation starts off the same by creating a new node. Then, we go through each of the nodes until we reach the end that points to the null using a while loop. After reaching the last node, it is set to the new node and then setting the previous of the new node to that current node.
Screenshot	
Discussion	The general insert operation works by creating a new node, then setting the next of that node to the next of the current node. Then, the previous of the new node as the current node. If the next of the new nodes is not null, then we set that node's previous to the new node. Then, we set the current node next as new node;

Screenshot	
Discussion	The delete a node operation works by firsts seeing if the list is empty if the head is a null. Then, we create a current Node and setting it to the address of the head. Then, a while loop is used to go through each of the node until it finds the node that contains the data that we want to delete. If the current node is equal to null, that means that it reached the end and it did not find a data. If the current node's previous is not the null, then we set that node's next to the next of the current node. If the current node's next is not equal to the null, we set that node's prev to the current node's prev.

```
85  template<typename T>
86  void dllDeleteNode(T findData, DoubleList<T>** head){
87      if(*head == nullptr){
88          std::cout<<"List is empty"<<std::endl;
89          return;
90      }
91
92      DoubleList<T>* currentNode = *head;
93
94      while(currentNode != nullptr && currentNode->data != findData){
95          currentNode = currentNode->next;
96      }
97
98      if(currentNode == nullptr){
99          std::cout<<"Data not found"<<std::endl;
100         return;
101     }
102
103     if(currentNode->prev != nullptr){
104         currentNode->prev->next = currentNode->next;
105     } else {
106         *head = currentNode->next;
107     }
108
109     if (currentNode->next != nullptr) {
110         currentNode->next->prev = currentNode->prev;
111     }
112
113     delete currentNode;
114
115 }
116
117 #endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Adding A Song

```
22  template<typename T>
23  void addSong(T newData, Playlist<T>** head){
24      Playlist<T>* newNode = createNode(newData);
25
26      if(*head == nullptr){
27          *head = newNode;
28          newNode->next = newNode;
29          newNode->prev = newNode;
30          return;
31      }
32
33      Playlist<T>* tail = (*head)->prev;
34
35      newNode->next = *head;
36      newNode->prev = tail;
37      tail->next = newNode;
38      (*head)->prev = newNode;
39      *head = newNode;
40
41      return;
42  }
```

Output

```
==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 1

Enter Song Title: Song1

Song Added Successfully!
```

Deleting a Song

```cpp
template<typename T>
void deleteSong(T findData, Playlist<T>** head){

    if(*head == nullptr){
        std::cout<<"The Playlist is Empty"<<std::endl;
        return;
    }

    Playlist<T>* currentNode = *head;

    do{
        if(currentNode->data == findData){
            break;
        }
        currentNode = currentNode->next;
    }while(currentNode != *head);

    if (currentNode->data != findData){
        std::cout<<"No Song Found"<<std::endl;
        return;
    }

    if(currentNode->next == currentNode){
        delete currentNode;
        *head = nullptr;
        return;
    }

    currentNode->prev->next = currentNode->next;
    currentNode->next->prev = currentNode->prev;

    if(currentNode == *head){
        *head = currentNode->next;
    }

    delete currentNode;
}
```

Output

```
==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 2

Enter Song Title to Remove: Song1

Song Removed Successfully!
```

Play All Songs

```
template<typename T>
void DisplaySongs(Playlist<T>* head){
    if(head == nullptr){
        std::cout<<"The Playlist is Empty"<<std::endl;
        return;
    }

    Playlist<T>* currentNode = head;

    do{
        std::cout<<currentNode->data<<std::endl;
        currentNode = currentNode->next;
    }while(currentNode != head);
    std::cout<<std::endl;
}
```

Output

```
==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 1

Enter Song Title: Song1

Song Added Successfully!

==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 1

Enter Song Title: Song2

Song Added Successfully!

==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 3

Song2
Song1
```

Next/Previous Song

```
template<typename T>
void nextSong(Playlist<T>** current){
    if(*current == nullptr){
        std::cout<<"\nPlaylist is Empty"<<std::endl;
        return;
    }

    *current = (*current)->next;
    std::cout<< "\nNow Playing: "<<(*current)->data<<std::endl;
}

template<typename T>
void previousSong(Playlist<T>** current){
    if(*current == nullptr){
        std::cout<<"Playlist is Empty"<<std::endl;
        return;
    }

    *current = (*current)->prev;
    std::cout<<"\nNow Playing: "<<(*current)->data<<std::endl;
}
#endif
```

Output

```
==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 5


Now Playing: Song2

==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 6


Now Playing: Song1
```

Current Song and Exit

```
case 4:
    if(current == nullptr){
        std::cout<<"\nNo Song Is Playing"<<std::endl;
    }
    else{
        std::cout<<"\nCurrently Playing: "<<current->data<<std::endl;
    }
    break;
```

```
case 7:
    std::cout<<"\nExiting!"<<std::endl;
    break;
```

Output

```
==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 4

Currently Playing: Song1

==== SPOOTIFY SONG PLAYLIST ====
1. Add Song
2. Remove a Song
3. Play All Songs
4. Play Current Song
5. Next Song
6. Previous Song
7. Exit
Choose a Feature: 7

Exiting!
```

Main.cpp

```cpp
#include <iostream>
#include <string>
#include "playlist.h"

int main(){

    Playlist<std::string>* head = nullptr;
    Playlist<std::string>* current = nullptr;

    int choice = 0;
    std::string song;

    while(choice != 7){
        std::cout<<"\n==== SPOOTIFY SONG PLAYLIST ===="<<std::endl;
        std::cout<<"1. Add Song"<<std::endl;
        std::cout<<"2. Remove a Song"<<std::endl;
        std::cout<<"3. Play All Songs"<<std::endl;
        std::cout<<"4. Play Current Song"<<std::endl;
        std::cout<<"5. Next Song"<<std::endl;
        std::cout<<"6. Previous Song"<<std::endl;
        std::cout<<"7. Exit"<<std::endl;
        std::cout<<"Choose a Feature: ";
        std::cin>>choice;

        switch(choice){
            case 1:
                std::cout<<"\nEnter Song Title: ";
                std::cin>>song;
                addSong(song, &head);

                if(current == nullptr){
                    current = head;
                }

                std::cout<<"\nSong Added Successfully!"<<std::endl;
                break;
```

```
37
38          case 2:
39              std::cout<<"\nEnter Song Title to Remove: ";
40              std::cin>>song;
41              deleteSong(song, &head);
42
43              if(head == nullptr){
44                  current = nullptr;
45              }
46
47              std::cout<<"\nSong Removed Successfully!"<<std::endl;
48              break;
49
50          case 3:
51              std::cout<<std::endl;
52              DisplaySongs(head);
53              break;
54
55          case 4:
56              if(current == nullptr){
57                  std::cout<<"\nNo Song Is Playing"<<std::endl;
58              }
59              else{
60                  std::cout<<"\nCurrently Playing: "<<current->data<<std::endl;
61              }
62              break;
63
64          case 5:
65              std::cout<<std::endl;
66              nextSong(&current);
67              break;
68
69          case 6:
70              std::cout<<std::endl;
71              previousSong(&current);
72              break;
```

```
73
74              case 7:
75                  std::cout<<"\nExiting!"<<std::endl;
76                  break;
77
78          }
79
80      }
81      return 0;
82  }
```

Conclusion: The activity was all about the double linked list. Similar to the single linked list, it allows the user to have a list with a head that connects to various nodes. Each node contains data and can point to the next. However, in the double linked list, each node also points to the previous node. This is helpful for making various projects where you need to store data in an organized way. In the activity, the single linked list was used to display an array of characters while the double linked list was used to create a music playlist manager. I found it unique to create that. I was a bit confused first as to how a double linked list that is closed/a loop would work but I figured it out. Instead of setting the previous of the head and the next of the last node to null, we simply loop it back by creating a new node. Overall, I found the activity fun and I felt like I learnt a lot. However, I still get confused to the sheer amount of code that is contained on the header file and should keep practicing.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.